# Reviewer Pack — Minimal Rank of Formal Groups and DPLL Search Tree Height

Entry 58d7474b72d0 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-22 03:55:56 UTC

## Minimal Rank of Formal Groups and DPLL Search Tree Height

**Entry ID**: `58d7474b72d0` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-22 03:55:56 UTC

### 1. Conjecture

**Field A** (mathematical branch): Formal Group Theory **Field B** (complexity object): Complexity Theory: DPLL Search Tree Complexity

**Statement**:

> {'s1': 'For every Boolean satisfiability instance with n variables, the minimal rank of the associated formal group is bounded by a function f(n) that depends polynomially on n.', 's2': 'This bound implies an upper limit on the height of the DPLL search tree for the instance, where f(n) bounds this height directly.', 's3': 'An instance with a formal group of minimal rank exceeding f(n) would serve as a counterexample to the conjecture.'}

**Rationale (proposer's reasoning)**:

> {'s1': 'Formal groups are algebraic structures that encode non-commutative operations and have been applied in various fields. Their minimal rank can provide insights into the complexity of computations.', 's2': 'The DPLL search tree height is a measure of the complexity of solving Boolean satisfiability problems. A connection between formal group theory and this complexity measure could reveal new structures or methods for solving SAT instances.', 's3': 'This bridge has potential to expose underlying structures that might lead to improved algorithms for SAT.'}

**Taxonomy category**: `AVG_TO_WORST_CASE` (status at proposal time: partially_alive)

### 2. Pre-registration (Popper-style)

**Hash (SHA-256 prefix)**: `4474dcccc1b57f20`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

**Acceptance criterion**:

> The conjecture is supported if for all n variables, the minimal rank of the formal group does not exceed f(n) and the ratio of DPLL search tree height to minimal rank is bounded by f(n). The conjecture is falsified if any instance with n variables has a formal group of minimal rank exceeding f(n) or the ratio exceeds f(n).

## 3. Barrier filter (F1)

**Final**: PASS

| Barrier | Verdict | Confidence | LLM₁ | LLM₂ |
|---|---|---|---|---|
| RELATIVIZATION | UNCERTAIN | 1.00 | HITS | SAFE |
| NATURAL_PROOFS | SAFE | 0.50 | UNCERTAIN | UNCERTAIN |
| ALGEBRIZATION | UNCERTAIN | 1.00 | UNCERTAIN | HITS |
| KARP_LIPTON | SAFE | 0.50 | UNCERTAIN | UNCERTAIN |

## 4. Novelty audit

**Verdict**: NOVEL against 1 hits across arXiv + Semantic Scholar + ECCC

**Search queries** (3): - `formal groups AND DPLL search tree complexity` - `minimal rank formal groups AND upper bound DPLL tree height` - `Boolean satisfiability instance AND polynomial bound minimal rank formal group`

**Top relevant hits considered**: - [s2:033d394ddc09c7770fbda5eefc631b4ea154f5fd] STACS 96 : 13th Annual Symposium on Theoretical Aspects of Computer Science, Grenoble, France, February 22-24, 1996 : pr

## 5. Empirical test harness

**Multi-seed protocol**: 5 seeds (11, 23, 37, 53, 71)
**Sandbox**: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

### 5.1 Generated Python source

```python
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def gaussian_elimination(A):
        m, n = len(A), len(A[0])
        for i in range(m):
            max_row = i + max(range(i, m), key=lambda r: abs(A[r][i]))
            A[i], A[max_row] = A[max_row], A[i]
            if A[i][i] == 0:
                continue
            for j in range(n):
                A[i][j] /= A[i][i]
            for k in range(m):
                if k != i and A[k][i] != 0:
                    factor = A[k][i]
                    for j in range(n):
                        A[k][j] -= factor * A[i][j]
        return A

    def matrix_multiply(A, B):
        m, n, p = len(A), len(B), len(B[0])
        C = [[0] * p for _ in range(m)]
```

```
    for i in range(m):
        for j in range(p):
            for k in range(n):
                C[i][j] += A[i][k] * B[k][j]
    return C

def construct_formal_group(instance):
    n = len(instance)
    G = [[0]*n for _ in range(n)]
    for clause in instance:
        for literal in clause:
            if literal > 0:
                G[literal-1][(literal-1+n)%n] += 1
            else:
                G[-literal-1][(literal-1+n)%n] -= 1
    return gaussian_elimination(G)

def dpll_search_tree_height(instance):
    n = len(instance)
    stack = [(0, [])]
    max_height = 0
    while stack:
        node, assignment = stack.pop()
        if node == n:
            max_height = max(max_height, len(assignment))
            continue
        for literal in instance[node]:
            new_assignment = assignment[:]
            if literal > 0:
                new_assignment.append(literal)
            else:
                new_assignment.append(-literal)
            stack.append((node + 1, new_assignment))
    return max_height

def minimal_rank(G):
    rank = 0
    for row in G:
        if any(row):
            rank += 1
    return rank

n = random.randint(5, 40)
instance = [[random.choice([-i, i]) for _ in range(n)] for _ in range(n)]

formal_group = construct_formal_group(instance)
minimal_rank_value = minimal_rank(formal_group)
dpll_height = dpll_search_tree_height(instance)

f_n = n**2  # Example polynomial function
ratio = dpll_height / minimal_rank_value

return {
    "metric_name": "Ratio of DPLL Search Tree Height to Minimal Rank",
    "metric_value": ratio,
    "instances_tested": 1,
```

```
        "conjecture_holds": ratio <= f_n,
        "counterexample": "" if ratio <= f_n else f"Instance with n={n} has minimal rank {minimal_
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2**i + 3 for i in range(5, 8)]  # Default to first

    results = []
    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result)

    mean_value = sum(result["metric_value"] for result in results) / len(results)
    std_value = math.sqrt(sum((result["metric_value"] - mean_value)**2 for result in results) / le
    support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

    if all(result["conjecture_holds"] for result in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_frac
    elif any(not result["conjecture_holds"] for result in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conj
        print(f"RESULT: FALSIFIED counterexample=\"Instance with n={n} has minimal rank {minimal_r
    else:
        print(f"RESULT: INCONCLUSIVE reason=unknown")
```

## 6. Per-seed results

*(no seed-level data — test crashed before producing TRIAL: lines)*

## 7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_bafa1e73.py", line 106, in <module>
    trial_result = run_trial(seed)
                   ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_bafa1e73.py", line 83, in run_trial
    instance = [[random.choice([-i, i]) for _ in range(n)] for _ in range(n)]
                                 ^
NameError: name 'i' is not defined. Did you mean: 'id'?
```

## 8. Critic adversarial review

**Critic verdict**: CHALLENGE

**Critic reasoning**:

> skipped: test crashed before producing data

## 9. Final verdict & safety rail

**Verdict**: INCONCLUSIVE

**Reasoning**:

The test crashed before producing data, which means we cannot verify if the conjecture's support conditions are met. | next: Re-run the test to ensure it completes without crashing and produces results.

## 11. Audit log (LLM calls)

**Total LLM calls**: 9

| # | Phase | Provider | Model | Tokens in | out | Latency (ms) |
|---|---|---|---|---|---|---|
| 1 | propose | ollama_remote | glm4:latest | 0 | 0 | 14816 |
| 2 | preregistration | ollama_remote | glm4:latest | 0 | 0 | 9161 |
| 3 | novelty | ollama_remote | glm4:latest | 0 | 0 | 8680 |
| 4 | novelty | ollama_remote | glm4:latest | 0 | 0 | 9203 |
| 5 | test_gen | ollama_remote | qwen2.5-coder:7b | 0 | 0 | 15991 |
| 6 | test_gen | ollama_remote | qwen2.5-coder:7b | 0 | 0 | 12695 |
| 7 | test_gen | ollama_remote | qwen2.5-coder:7b | 0 | 0 | 10002 |
| 8 | test_gen | ollama_remote | qwen2.5-coder:7b | 0 | 0 | 11878 |
| 9 | judge | ollama_remote | glm4:latest | 0 | 0 | 11825 |

**Totals**: 0 input tokens, 0 output tokens, ~$0.0000 cost, 104252 ms total latency. Provider mix: {'ollama_remote': 9}

*(full prompt+response transcripts available in `research/audit/58d7474b72d0.jsonl`)*

## 12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/58d7474b72d0.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

**Sandbox file**: `research/pvsnp_sandbox/test_*.py` (per-cycle)
**Replay tarball**: `research/replay/58d7474b72d0.tar.gz` (if generated)